

MECHATRONICS AND IOT

ASSIGNMENT REPORT - 11

TITLE:

Design and develop a Cloud-Based Industrial DG Monitoring and Analytics System. The system collects and analyzes diesel generator performance data using cloud technologies for remote monitoring applications.

DONE BY ,

HARI KRISHNA JAGADEESH-24MER011

HARISH DHARMALINGAM – 24MER014

RANJITH KUMAR P – 24MER048

RISHI KESAV A – 24MER049

VENKATRAMANAN B – 24MER061

Project Overview:

The Industrial Diesel Generator Monitoring and Maintenance System is an IoT-based system designed to continuously monitor the operating condition and performance of diesel generators. The system uses sensors to measure important parameters such as engine temperature, battery voltage, fuel level and vibration, which are processed by a microcontroller.

The collected parameters are displayed locally on an OLED/LCD display, allowing operators to monitor the generator's condition in real time. The system compares the measured values with predefined safety and operating limits and generates alerts when abnormal conditions are detected.

The system can also use Wi-Fi connectivity to transmit generator performance data to a cloud platform for remote monitoring, data logging and analysis. By tracking operating parameters and maintenance-related information, the system supports preventive maintenance, helps identify potential faults at an early stage and reduces unexpected generator downtime.

The proposed system provides a low-cost, real-time and automated solution for improving diesel generator reliability, operational safety and maintenance efficiency in industrial applications.

Problem Statement:

Industrial diesel generators are critical backup power sources, and their reliable operation depends on continuous monitoring and timely maintenance. Conventional maintenance practices often rely on manual inspection and fixed maintenance schedules, making it difficult to identify developing faults at an early stage.

Parameters such as engine temperature, battery voltage, fuel level and vibration can change during generator operation. Failure to monitor these parameters continuously may result in unexpected breakdowns, increased maintenance costs, reduced generator efficiency and unplanned downtime.

Therefore, there is a need for a real-time generator monitoring and maintenance system that can continuously track important operating parameters, detect abnormal conditions and provide timely alerts. Such a system can support preventive maintenance, improve generator reliability and enhance operational safety in industrial applications.

Proposed Solution:

The proposed system uses an ESP32 microcontroller and multiple sensors to continuously monitor the operating condition of an industrial diesel generator. Sensors are used to measure important parameters such as engine temperature, battery voltage, fuel level and vibration.

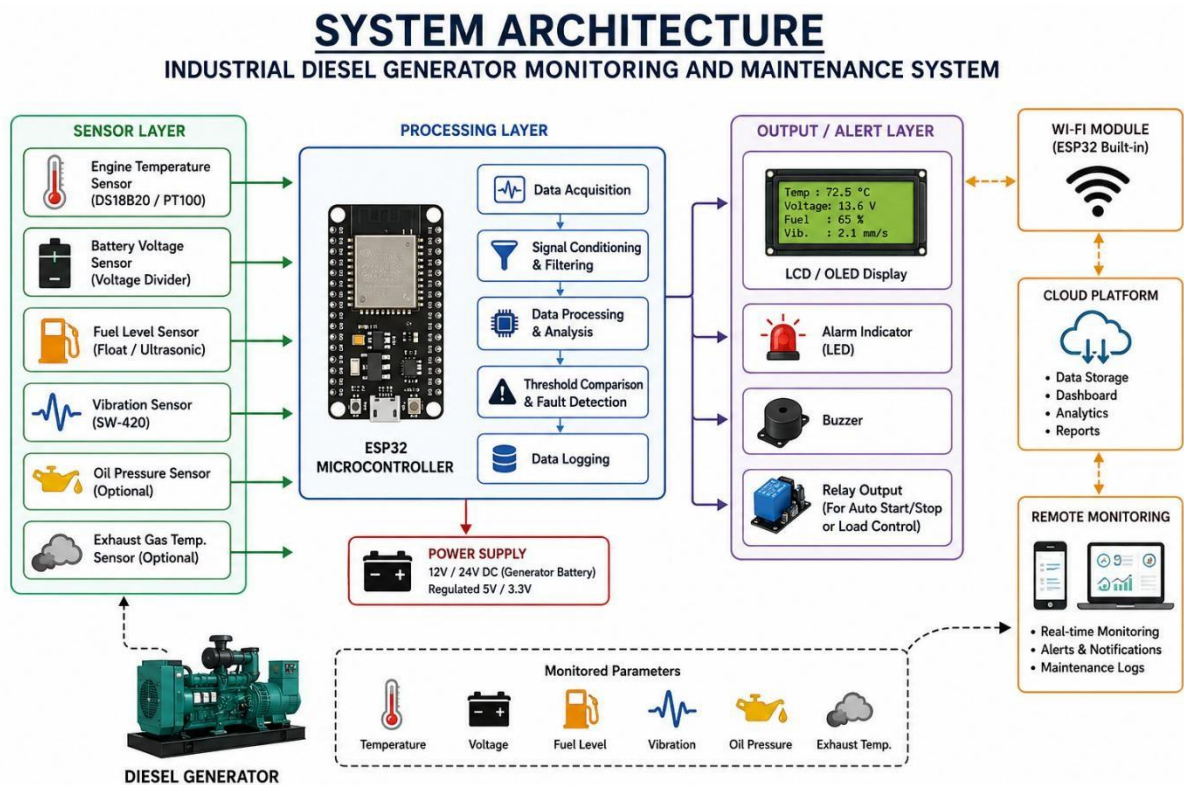
The ESP32 collects and processes the sensor readings and compares them with predefined operating limits. The current generator parameters are displayed on an OLED/LCD display, allowing operators to monitor the generator locally in real time.

When any parameter exceeds its safe operating range, the system activates a buzzer and warning indicator to alert the operator. The system can also record operating conditions and maintenance-related information to help identify abnormal trends and support preventive maintenance.

Using the ESP32's built-in Wi-Fi connectivity, the monitored data can be transmitted to an IoT/cloud platform for remote monitoring, data logging and performance analysis.

The proposed solution provides a low-cost, real-time and automated monitoring system that helps detect potential generator faults early, reduce unexpected downtime and improve generator reliability, safety and maintenance efficiency.

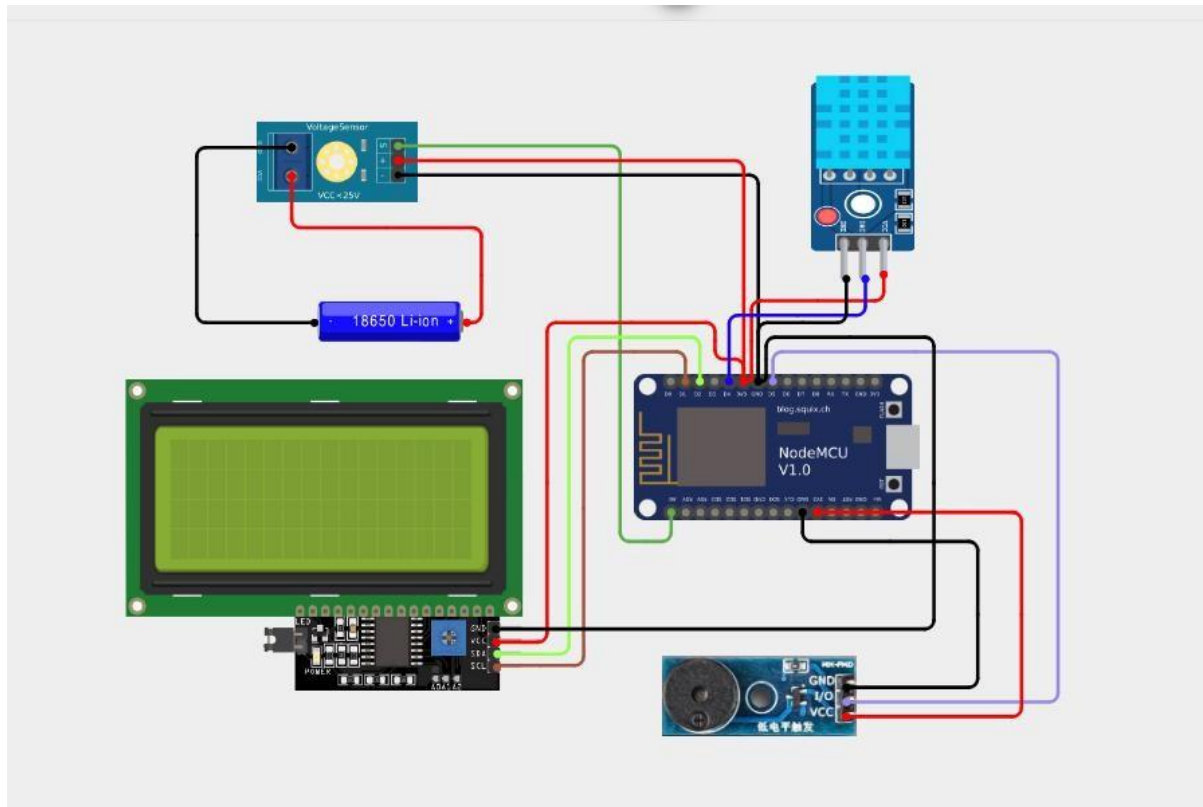
System Architecture:



Circuit Table :

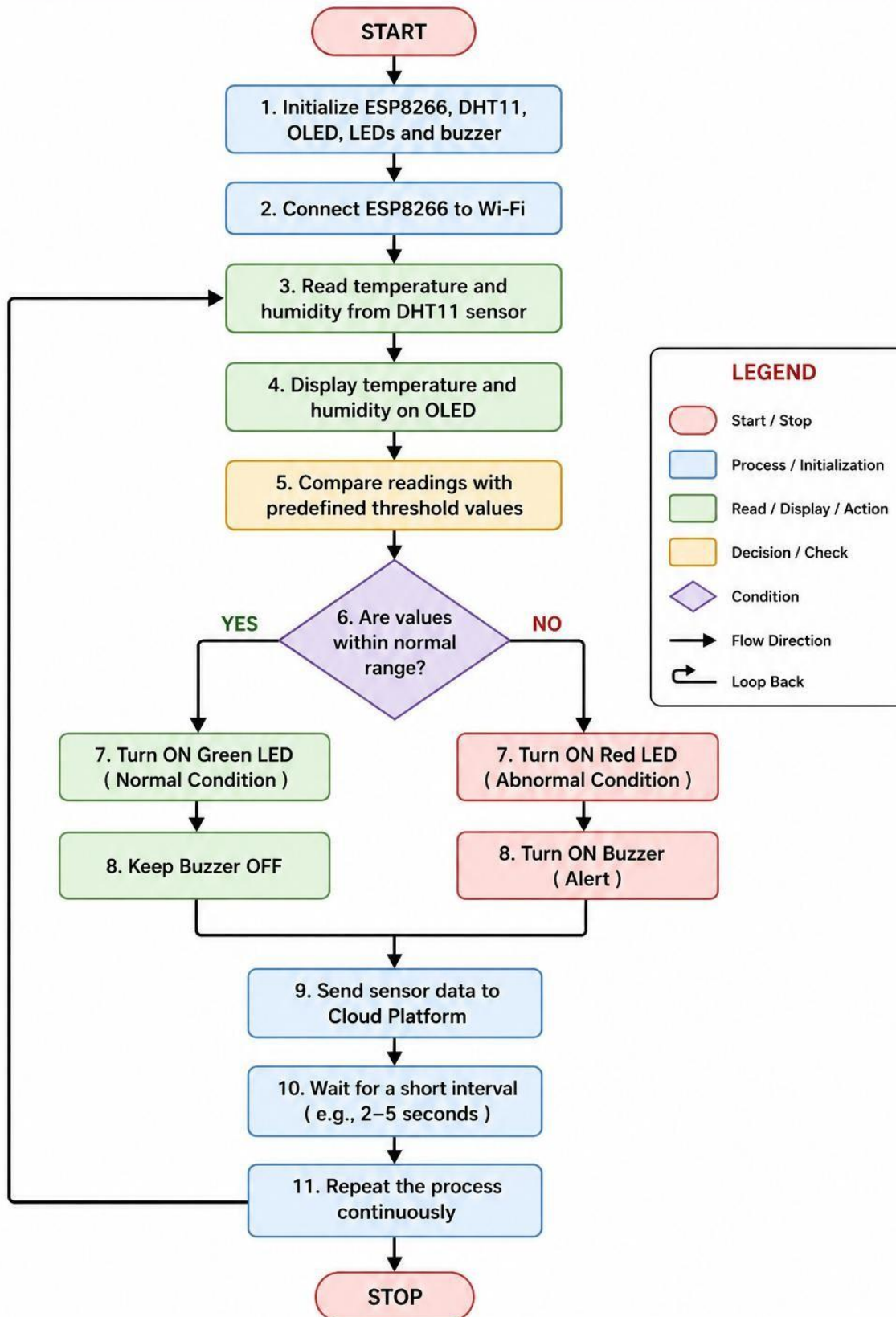
Component	Pin/Terminal	ESP8266 Connection	Purpose
DHT11 Sensor	VCC	3.3V	Power supply
LCD Display	VCC	3.3V	Safe indication
ESP8266	VIN/USB	5V USB	Hazardous indication
Buzzer	Positive (+)	GPIO 14	Alert
Voltage Sensor	S	GPIO 34 (ADC)	Battery voltage measurement
Battery	Positive (+)	Voltage Sensor Input (+)	Battery voltage source

Circuit Design:



Algorithm:

ALGORITHM – SMART REFRIGERATOR MONITORING SYSTEM



Coding:

```
// =====  
  
// INDUSTRIAL DIESEL GENERATOR MONITORING SYSTEM  
  
// ESP8266 NODEMCU  
  
//  
  
// COMPONENTS:  
  
// 1. ESP8266 NodeMCU  
  
// 2. DHT11  
  
// 3. Voltage Sensor  
  
// 4. OLED Display  
  
// 5. Buzzer  
  
//  
  
// ADAFRUIT IO FEEDS:  
  
// 1. temperature  
  
// 2. voltage  
  
// 3. runtime-gen  
  
// 4. humidity  
  
// 5. generator-status  
  
//  
  
// =====  
  
#include <ESP8266WiFi.h>  
  
#include "Adafruit_MQTT.h"  
  
#include "Adafruit_MQTT_Client.h"  
  
#include <DHT.h>  
  
#include <Wire.h>  
  
#include <Adafruit_GFX.h>  
  
#include <Adafruit_SSD1306.h>  
  
#include <EEPROM.h>  
  
// =====
```

```
// WIFI SETTINGS

// =====

const char* ssid = "Hari";

const char* password = "11111111";

// =====

// ADAFRUIT IO SETTINGS

// =====

#define AIO_SERVER    "io.adafruit.com"

#define AIO_SERVERPORT 1883

#define AIO_USERNAME  "Hari87789"

// IMPORTANT:

// Put your current Adafruit IO key here.

#define AIO_KEY        "aio_xQtL08wRGFwdE9Avb1GQd2hYbady"

// =====

// WIFI CLIENT

// =====

WiFiClient client;

// =====

// MQTT CLIENT

// =====

Adafruit_MQTT_Client mqtt(
  &client,
  AIO_SERVER,
  AIO_SERVERPORT,
  AIO_USERNAME,
  AIO_KEY
);

// =====

// ADAFRUIT IO FEEDS

// =====
```



```
Adafruit_MQTT_Publish temperatureFeed =
```

```
Adafruit_MQTT_Publish(
```

```
    &mqtt,
```

```
    AIO_USERNAME "/feeds/temperature"
```

```
);
```

```
Adafruit_MQTT_Publish voltageFeed =
```

```
Adafruit_MQTT_Publish(
```

```
    &mqtt,
```

```
    AIO_USERNAME "/feeds/voltage"
```

```
);
```

```
Adafruit_MQTT_Publish runtimeFeed =
```

```
Adafruit_MQTT_Publish(
```

```
    &mqtt,
```

```
    AIO_USERNAME "/feeds/runtime-gen"
```

```
);
```

```
Adafruit_MQTT_Publish humidityFeed =
```

```
Adafruit_MQTT_Publish(
```

```
    &mqtt,
```

```
    AIO_USERNAME "/feeds/humidity"
```

```
);
```

```
Adafruit_MQTT_Publish generatorStatusFeed =
```

```
Adafruit_MQTT_Publish(
```

```
    &mqtt,
```

```
    AIO_USERNAME "/feeds/generator-status"
```

```
);
```

```
// =====
```

```
// ESP8266 PIN CONFIGURATION
```

```
// =====
```

```
//
```

```
// D4 = GPIO2
```

```
// D5 = GPIO14
// D2 = GPIO4
// D1 = GPIO5
//
// Using GPIO numbers directly avoids:
// 'D4 was not declared in this scope'
// =====
#define DHT_PIN 2
#define DHT_TYPE DHT11
#define VOLTAGE_SENSOR_PIN A0
#define BUZZER_PIN 14
#define OLED_SDA 4
#define OLED_SCL 5
// =====
// DHT11
// =====

DHT dht(
    DHT_PIN,
    DHT_TYPE
);
// =====

// OLED
// =====

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(
    SCREEN_WIDTH,
    SCREEN_HEIGHT,
    &Wire,
    -1
```

```

);
// =====
// EEPROM
// =====
#define EEPROM_SIZE 32
// =====
// VOLTAGE SETTINGS
// =====
//
// Your maximum voltage is approximately 3.5 V.
//
// Therefore we are NOT multiplying the ADC voltage
// by 5.
//
// IMPORTANT:
// Keep the voltage at ESP8266 A0 within the safe range
// of your particular NodeMCU board.
// =====
const float A0_MAX_VOLTAGE = 3.3;
const int ADC_MAX = 1023;
// =====
// GENERATOR STATUS THRESHOLD
// =====
//
// Generator OFF:
// Voltage < 0.5 V
//
// Generator ON:
// Voltage >= 0.5 V
// =====

```

```
const float GENERATOR_ON_VOLTAGE = 0.5;

// =====

// BUZZER THRESHOLDS

// =====

const float HIGH_TEMPERATURE = 50.0;

const float HIGH_VOLTAGE = 3.2;

const float LOW_VOLTAGE = 0.5;

// =====

// VARIABLES

// =====

float temperature = 0.0;

float humidity = 0.0;

float voltage = 0.0;

// Runtime stored in minutes

float runtimeMinutes = 0.0;

bool generatorRunning = false;

// =====

// TIMERS

// =====

unsigned long lastSensorRead = 0;

unsigned long lastUpload = 0;

unsigned long lastRuntimeUpdate = 0;

unsigned long lastEEPROMSave = 0;

unsigned long lastDisplayUpdate = 0;

// =====

// OLED PAGE

// =====

int displayPage = 0;

// =====

// WIFI CONNECTION
```

```
// =====  
  
void connectWiFi()  
{  
    Serial.println();  
  
    Serial.print(  
        "Connecting to WiFi"  
    );  
    WiFi.mode(WIFI_STA);  
    WiFi.begin(  
        ssid,  
        password  
    );  
    int attempts = 0;  
    while (  
        WiFi.status() != WL_CONNECTED  
    )  
    {  
        delay(500);  
        Serial.print(".");  
        attempts++;  
        if (attempts >= 40)  
        {  
            Serial.println();  
            Serial.println(  
                "WiFi connection failed!"  
            );  
            return;  
        }  
    }  
}
```

```
Serial.println();

Serial.println(
    "WiFi Connected!"
);
Serial.print(
    "IP Address: "
);
Serial.println(
    WiFi.localIP()
);
Serial.print(
    "Signal Strength: "
);
Serial.print(
    WiFi.RSSI()
);
Serial.println(
    " dBm"
);
}

// =====

// ADAFRUIT IO CONNECTION

// =====

void connectMQTT()
{
    if (mqtt.connected())
    {
        return;
    }
}
```

```
Serial.print(
  "Connecting to Adafruit IO"
);
int8_t ret;
int attempts = 0;
while (
  (ret = mqtt.connect()) != 0
)
{
  Serial.print(".");
  mqtt.disconnect();
  delay(5000);
  attempts++;
  if (attempts >= 5)
  {
    Serial.println();
    Serial.println(
      "Adafruit IO connection failed!"
    );
    return;
  }
}
Serial.println();
Serial.println(
  "Adafruit IO Connected!"
);
}
// =====
// READ DHT11
// =====
```

```

void readDHT()
{
    float t =
    dht.readTemperature();
    float h =
    dht.readHumidity();
    if (!isnan(t))
    {
        temperature = t;
    }
    if (!isnan(h))
    {
        humidity = h;
    }
}

// =====

// READ VOLTAGE

// =====

//

// Reads A0 and converts ADC reading to voltage.

//

// ESP8266 ADC:

// 0    = approximately 0 V
// 1023  = approximately 3.3 V

//

// =====

float readVoltage()
{
    long total = 0;
    const int samples = 50;

```



```

for (
    int i = 0;
    i < samples;
    i++
)
{
    total += analogRead(
        VOLTAGE_SENSOR_PIN
    );
    delay(2);
}

float averageADC =
(float)total / samples;
float measuredVoltage =
(
    averageADC *
    A0_MAX_VOLTAGE
) / ADC_MAX;

// Remove tiny noise
if (
    measuredVoltage < 0.05
)
{
    measuredVoltage = 0.0;
}

return measuredVoltage;
}

// =====

// CHECK GENERATOR STATUS

// =====

```

```
void checkGeneratorStatus()
{
    if (
        voltage >=
        GENERATOR_ON_VOLTAGE
    )
    {
        generatorRunning = true;
    }
    else
    {
        generatorRunning = false;
    }
}

// =====
// UPDATE GENERATOR RUNTIME
// =====
//
// Runtime increases only while generator is RUNNING.
//
// Runtime is stored in MINUTES.
// =====

void updateRuntime()
{
    unsigned long now =
        millis();
    if (
        lastRuntimeUpdate == 0
    )
    {
```

```

    lastRuntimeUpdate = now;

    return;
}

unsigned long elapsed =

    now -

    lastRuntimeUpdate;
if (generatorRunning)
{
    float elapsedMinutes =

    elapsed / 60000.0;

    runtimeMinutes +=

    elapsedMinutes;
}

lastRuntimeUpdate =

    now;
}

// =====
// SAVE RUNTIME
// =====
//
// Saves runtime every 60 seconds.
// =====

void saveRuntime()
{
    if (

        millis() -

        lastEEPROMSave >= 60000

    )
    {
        EEPROM.put(

```

```

    0,
    runtimeMinutes
);
EEPROM.commit();
lastEEPROMSave =
millis();
Serial.print(
    "Runtime saved: "
);
Serial.print(
    runtimeMinutes
);
Serial.println(
    " minutes"
);
}
}
// =====
// BUZZER CONTROL
// =====
//
// Buzzer ON when:
// Temperature > 50 C
// OR voltage > 3.2 V
//
// =====

void controlBuzzer()
{
    bool warning = false;
    // High temperature

```

```
if (
    temperature >=
    HIGH_TEMPERATURE
)
{
    warning = true;
}
// High voltage
if (
    generatorRunning &&
    voltage >= HIGH_VOLTAGE
)
{
    warning = true;
}
if (warning)
{
    digitalWrite(
        BUZZER_PIN,
        HIGH
    );
}
else
{
    digitalWrite(
        BUZZER_PIN,
        LOW
    );
}
}
```

```
// =====  
// OLED PAGE 1  
// =====  
  
void showPage1()  
{  
    display.clearDisplay();  
    display.setTextColor(  
        SSD1306_WHITE  
    );  
    display.setTextSize(1);  
    display.setCursor(  
        0,  
        0  
    );  
    display.println(  
        "DIESEL GENERATOR"  
    );  
    display.drawLine(  
        0,  
        10,  
        127,  
        10,  
        SSD1306_WHITE  
    );  
    // Temperature  
    display.setCursor(  
        0,  
        18  
    );  
    display.print(  

```

```
"Temp: "
);
display.print(
    temperature,
    1
);
display.println(
    " C"
);
// Humidity
display.setCursor(
    0,
    30
);
display.print(
    "Hum : "
);
display.print(
    humidity,
    0
);
display.println(
    " %"
);
// Voltage
display.setCursor(
    0,
    42
);
display.print(
```

```
"Volt: "  
);  
display.print(  
    voltage,  
    2  
);  
display.println(  
    " V"  
);  
// Generator status  
display.setCursor(  
    0,  
    54  
);  
display.print(  
    "Status: "  
);  
if (generatorRunning)  
{  
    display.println(  
        "RUNNING"  
    );  
}  
else  
{  
    display.println(  
        "STOPPED"  
    );  
}  
display.display();
```



```
}  
  
// =====  
  
// OLED PAGE 2  
  
// =====  
  
void showPage2()  
{  
    display.clearDisplay();  
    display.setTextColor(  
        SSD1306_WHITE  
    );  
    display.setTextSize(1);  
    display.setCursor(  
        0,  
        0  
    );  
    display.println(  
        "GENERATOR MONITOR"  
    );  
    display.drawLine(  
        0,  
        10,  
        127,  
        10,  
        SSD1306_WHITE  
    );  
    // Voltage  
    display.setCursor(  
        0,  
        20  
    );  
}
```

```
display.print(  
    "Voltage: "  
);  
display.print(  
    voltage,  
    2  
);  
display.println(  
    " V"  
);  
// Runtime  
display.setCursor(  
    0,  
    34  
);  
display.print(  
    "Time: "  
);  
display.print(  
    runtimeMinutes,  
    1  
);  
display.println(  
    " min"  
);  
// Status  
display.setCursor(  
    0,  
    48  
);
```

```
if (generatorRunning)
{
    display.println(
        "STATUS: RUNNING"
    );
}
else
{
    display.println(
        "STATUS: STOPPED"
    );
}
display.display();
}

// =====
// UPDATE OLED
// =====

void updateOLED()
{
    if (
        millis() -
        lastDisplayUpdate < 3000
    )
    {
        return;
    }
    lastDisplayUpdate =
    millis();
    displayPage++;
}
```

```
if (displayPage > 1)
{
    displayPage = 0;
}
if (displayPage == 0)
{
    showPage1();
}
else
{
    showPage2();
}
}
// =====
// SEND DATA TO ADAFRUIT IO
// =====
//
// Upload every 30 seconds.
//
// Only 5 feeds are used.
// =====

void sendToAdafruit()
{
    connectMQTT();
    if (!mqtt.connected())
    {
        return;
    }
    Serial.println();
    Serial.println(
```

```
"Uploading data to Adafruit IO..."

);
// -----
// TEMPERATURE
// -----
if(
  temperatureFeed.publish(
    temperature
  )
)
{
  Serial.println(
    "Temperature: OK"
  );
}
else
{
  Serial.println(
    "Temperature: FAILED"
  );
}
delay(200);
// -----
// VOLTAGE
// -----
if(
  voltageFeed.publish(
    voltage
  )
)
{
```

```
Serial.println(
    "Voltage: OK"
);
}
else
{
    Serial.println(
        "Voltage: FAILED"
    );
}
delay(200);
// -----
// RUNTIME
// -----
if(
    runtimeFeed.publish(
        runtimeMinutes
    )
)
{
    Serial.println(
        "Runtime: OK"
    );
}
else
{
    Serial.println(
        "Runtime: FAILED"
    );
}
```

```
delay(200);

// -----

// HUMIDITY

// -----

if(
    humidityFeed.publish(
        humidity
    )
)
{
    Serial.println(
        "Humidity: OK"
    );
}
else
{
    Serial.println(
        "Humidity: FAILED"
    );
}

delay(200);

// -----

// GENERATOR STATUS

// -----

if (generatorRunning)
{
    if (
        generatorStatusFeed.publish(
            "RUNNING"
        )
    )
}
```

```
)  
{  
    Serial.println(  
        "Generator Status: RUNNING"  
    );  
}  
else  
{  
    Serial.println(  
        "Generator Status: FAILED"  
    );  
}  
}  
else  
{  
    if (  
        generatorStatusFeed.publish(  
            "STOPPED"  
        )  
    )  
    {  
        Serial.println(  
            "Generator Status: STOPPED"  
        );  
    }  
    else  
    {  
        Serial.println(  
            "Generator Status: FAILED"  
        );  
    }  
}
```



```
    }  
}  
Serial.println();  
Serial.println(  
  "Upload complete!"  
);  
}  
// =====  
// SERIAL MONITOR  
// =====  
void printData()  
{  
  Serial.println();  
  Serial.println(  
    "===== "  
  );  
  Serial.println(  
    " INDUSTRIAL DIESEL GENERATOR MONITOR "  
  );  
  Serial.println(  
    "===== "  
  );  
  Serial.print(  
    "Temperature : "  
  );  
  Serial.print(  
    temperature  
  );  
  Serial.println(  
    " C"
```

```
);  
Serial.print(  
    "Humidity  : "  
);  
Serial.print(  
    humidity  
);  
Serial.println(  
    " %"  
);  
Serial.print(  
    "Voltage  : "  
);  
Serial.print(  
    voltage  
);  
Serial.println(  
    " V"  
);  
Serial.print(  
    "Time      : "  
);  
Serial.print(  
    runtimeMinutes  
);  
Serial.println(  
    " minutes"  
);  
Serial.print(  
    "Generator : "
```

```
);  
if (generatorRunning)  
{  
  Serial.println(  
    "RUNNING"  
  );  
}  
else  
{  
  Serial.println(  
    "STOPPED"  
  );  
}  
Serial.println(  
  "=====  
);  
}  
// =====  
// SETUP  
// =====  
  
void setup()  
{  
  Serial.begin(  
    115200  
  );  
  delay(1000);  
  Serial.println();  
  Serial.println(  
    "=====  
);
```

```
Serial.println(
    " INDUSTRIAL DIESEL GENERATOR SYSTEM"
);
Serial.println(
    " ESP8266 NODEMCU"
);
Serial.println(
    "===== "
);
// -----
// BUZZER
// -----
pinMode(
    BUZZER_PIN,
    OUTPUT
);
digitalWrite(
    BUZZER_PIN,
    LOW
);
// -----
// DHT11
// -----
dht.begin();
// -----
// OLED
// -----
Wire.begin(
    OLED_SDA,
    OLED_SCL
```

```
);  
if (  
    !display.begin(  
        SSD1306_SWITCHCAPVCC,  
        0x3C  
    )  
)  
{  
    Serial.println(  
        "OLED NOT FOUND!"  
    );  
    while (true)  
    {  
        delay(1000);  
    }  
}  
display.clearDisplay();  
display.setTextColor(  
    SSD1306_WHITE  
);  
display.setTextSize(1);  
display.setCursor(  
    5,  
    15  
);  
display.println(  
    "DIESEL GENERATOR"  
);  
display.setCursor(  
    15,
```

```
    30
);
display.println(
    "MONITORING"
);
display.setCursor(
    30,
    45
);
display.println(
    "STARTING..."
);
display.display();
delay(2000);
// -----
// EEPROM
// -----

EEPROM.begin(
EEPROM_SIZE
);
EEPROM.get(
0,
    runtimeMinutes
);
// Check for invalid data
if (
    isnan(runtimeMinutes) ||
    runtimeMinutes < 0 ||
    runtimeMinutes > 1000000
)
```

```
{
  runtimeMinutes = 0;
}
Serial.print(
  "Saved Runtime: "
);
Serial.print(
  runtimeMinutes
);
Serial.println(
  " minutes"
);
// -----
// WIFI
// -----
connectWiFi();
// -----
// ADAFRUIT IO
// -----
connectMQTT();
// -----
// RUNTIME TIMER
// -----

lastRuntimeUpdate =
millis();
Serial.println();
Serial.println(
  "SYSTEM READY!"
);
}
```

```
// =====  
// MAIN LOOP  
// =====  
void loop()  
{  
  // =====  
  // READ SENSORS EVERY 2 SECONDS  
  // =====  
  if (  
    millis() -  
    lastSensorRead >= 2000  
  )  
  {  
    lastSensorRead =  
      millis();  
    // DHT11  
    readDHT();  
    // Voltage  
    voltage =  
      readVoltage();  
    // Generator status  
    checkGeneratorStatus();  
    // Runtime  
    updateRuntime();  
    // Buzzer  
    controlBuzzer();  
    // Serial output  
    printData();  
  }  
  // =====
```



```
// OLED

// =====

updateOLED();

// =====

// SAVE RUNTIME

// =====

saveRuntime();

// =====

// ADAFRUIT IO

// =====

if (
    millis() -
    lastUpload >= 30000
)
{
    lastUpload =
    millis();
    sendToAdafruit();
}

// =====

// MQTT PROCESSING

// =====

if (mqtt.connected())
{
    mqtt.processPackets(10);
}

delay(10);
}
```

System Working:

- The Industrial Diesel Generator Monitoring and Maintenance System uses an ESP32 microcontroller and multiple sensors to continuously monitor important generator parameters such as engine temperature, battery voltage, fuel level and vibration.
- The sensors collect real-time data and send it to the ESP32 for processing. The ESP32 compares each parameter with predefined safe operating limits and displays the readings on an OLED/LCD display.
- When all parameters are within the safe range, the system indicates normal operation using a green LED. If any parameter exceeds its predefined limit, the red LED and buzzer are activated to alert the operator about a possible fault or abnormal condition.
- The system can also record the monitored parameters with timestamps and transmit the data through Wi-Fi to a cloud platform for remote monitoring and analysis. The recorded data helps identify abnormal operating trends and supports preventive maintenance.
- The system continuously repeats the monitoring process, helping improve generator reliability, reduce unexpected breakdowns and minimize maintenance downtime.

Bill Of Material:

S. no	Component	Specification	Quantity	Cost
1	ESP8266	Node MCU ESP8266	1	Rs. 450
2	DHT11 Sensor	Temperature & Humidity Sensor	1	Rs. 100
3	Voltage Sensor	0–25V DC Voltage Sensor	1	Rs. 50
4	Bread Board	Standard 830-point	1	Rs. 100
5	Jumper Wire	Male-Male / Male-Female	1 set	Rs. 50
6	Battery	12V Rechargeable Battery	1	Rs. 500
		Total		Rs. 1250

Prototype Implementation:

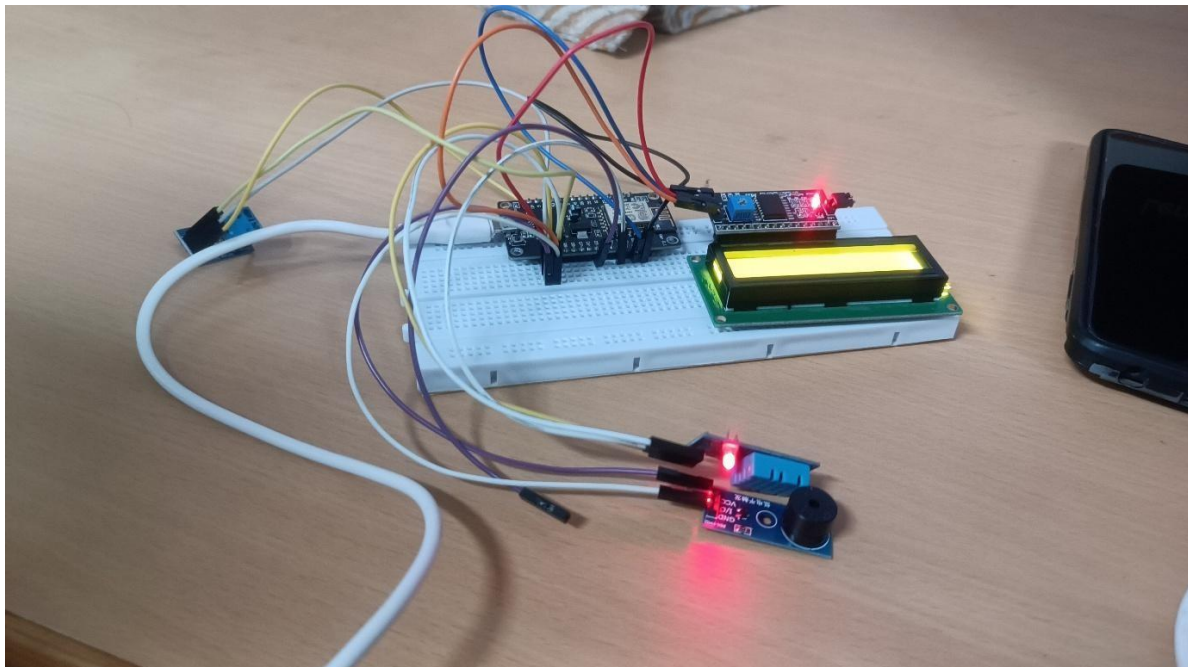
- The prototype was implemented using an ESP32 microcontroller, DHT11 sensor, voltage sensor, 12V battery, OLED display, LEDs, buzzer and relay module. The DHT11 sensor monitors the temperature and humidity around the generator, while the voltage sensor measures the battery voltage.
- The ESP32 collects and processes the sensor readings and displays the temperature, humidity and battery voltage on the OLED display. The measured values are compared with predefined threshold limits to identify abnormal generator conditions.
- During normal operation, the green LED indicates that the monitored parameters are within the acceptable range. If the temperature becomes too high or the battery voltage falls below the defined limit, the red LED and buzzer are activated to alert the operator.
- The ESP32's built-in Wi-Fi can also be used to transmit the generator parameters to an IoT/cloud platform for remote monitoring and data logging.

Prototype

flow:

DHT11 + Voltage Sensor + Battery → ESP32 → Data Processing → OLED Display + LED/Buzzer → Wi-Fi/Cloud

The prototype demonstrates a low-cost and real-time generator monitoring system that can support early fault detection and preventive maintenance.



Results and Performance:

- The developed Industrial Diesel Generator Monitoring and Maintenance System successfully monitored the generator's temperature, humidity and battery voltage using the DHT11 sensor and voltage sensor connected to the ESP32. The sensor readings were processed in real time and displayed on the OLED display.
- The system successfully compared the measured parameters with predefined operating limits. Under normal conditions, the green LED indicated proper operation. When the temperature exceeded the set limit or the battery voltage dropped below the defined threshold, the red LED and buzzer provided an immediate warning to the operator.

Performance Analysis

Parameter	Result
Temperature Monitoring	Successfully measured using DHT11
Humidity Monitoring	Successfully measured using DHT11
Battery Voltage Monitoring	Successfully measured using voltage sensor
Local Display	Real-time OLED display
Normal Condition	Green LED ON
Abnormal Condition	Red LED + Buzzer ON
Response	Real-time
Preventive Maintenance Support	Abnormal conditions can be detected early
Remote Monitoring	Wi-Fi/Cloud integration supported

Overall, the prototype demonstrated a low-cost, real-time and reliable generator monitoring solution. Continuous monitoring of temperature and battery voltage can help identify abnormal

operating conditions at an early stage, supporting preventive maintenance, reducing unexpected downtime and improving generator reliability and safety.

